

PCCA2 Project Report

Real Root Isolation

Encadrant: Edern Gillot

Hongjing ZHU

Sorbonne Université
Master Informatique M1 – CCA

1 Introduction

The problem of computing and isolating the real roots of a univariate polynomial is a fundamental question in computer algebra. Given a polynomial with real or rational coefficients, one aims to determine all its real roots and to enclose each of them in a disjoint interval with rational endpoints. This process is known as *real root isolation*.

Real root isolation plays an important role in various domains such as symbolic computation, robotics, control theory, and computational geometry. Many applications require not only approximating roots but also guaranteeing their existence and uniqueness within certified intervals.

In this project, we study and implement two classical approaches based on Sturm theory: the *Sturm sequence* and the *Sturm–Habicht sequence*. These methods allow us to count the number of real roots of a polynomial within a given interval by analyzing sign variations.

The main objectives of this project are:

- to implement real root isolation algorithms in C;
- to compare a naive implementation based on Sturm sequences with a more advanced approach using Sturm–Habicht sequences.

2 Theoretical Background

2.1 Notation and Real Root Isolation

Throughout this report, we denote by $\mathbb{Q}$ the field of rational numbers and by $\mathbb{R}$ the field of real numbers.

Let $P(x) \in \mathbb{Q}[x]$ be a univariate polynomial. The problem of *real root isolation* consists in computing a finite set of pairwise disjoint intervals with rational endpoints such that each interval contains exactly one real root of P , and every real root of P belongs to one of these intervals. This is the standard notion of real root isolation used in real algebraic geometry; see [1, Chapter 10].

To isolate the real roots of P , one first needs a method to count the number of real roots in a given interval. This is achieved using Sturm-type methods.

2.2 Sturm Sequences

Let $P \in \mathbb{R}[x]$ be a nonzero polynomial. The Sturm sequence associated with P is the finite sequence $(S_0, S_1, \dots, S_t)$ defined by:

$$S_0 = P, \quad S_1 = P',$$

and for every $i \geq 1$,

$$S_{i+1} = -\text{rem}(S_{i-1}, S_i),$$

where $\text{rem}(S_{i-1}, S_i)$ denotes the remainder of the Euclidean division of S_{i-1} by S_i .

The associated signed remainder sequence is the classical Sturm sequence; see [1, Section 2.2.2]. The construction stops when a constant polynomial is reached.

2.3 Sign Variations

Let $(a_0, a_1, \dots, a_m)$ be a finite sequence of real numbers. In order to define sign variations, we first remove all zero elements from the sequence and obtain a sequence $(b_0, b_1, \dots, b_k)$ of nonzero terms.

The number of sign variations of the original sequence, denoted by

$$V(a_0, a_1, \dots, a_m),$$

is defined as follows:

- if the sequence contains no nonzero element, then $V(a_0, \dots, a_m) = 0$;
- otherwise, $V(a_0, \dots, a_m)$ is the number of indices $j \in \{0, \dots, k-1\}$ such that $b_j \cdot b_{j+1} < 0$.

In other words, a sign variation corresponds to a change of sign between two consecutive nonzero elements. Zero values are ignored when counting sign variations, following the standard convention used in real root counting; see [1, Section 2.2].

2.4 Sturm Theorem

Let $a < b$ be two real numbers such that $P(a) \neq 0$ and $P(b) \neq 0$. Let

$$V_P(x) = V(S_0(x), S_1(x), \dots, S_t(x))$$

be the number of sign variations in the evaluations of the Sturm sequence at x .

Then, the number of distinct real roots of P in the interval (a, b) is given by:

$$V_P(a) - V_P(b).$$

This is precisely Sturm's theorem [1, Theorem 2.50], and it provides an exact method to count real roots in any interval whose endpoints are not roots of the polynomial.

2.5 A Root Bound

In order to apply interval subdivision algorithms, one first needs a finite interval that contains all real roots of the polynomial. A classical result is given by the Cauchy bound.

Let

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0 \quad \text{with } a_n \neq 0.$$

Then every complex root z of P satisfies:

$$|z| \leq 1 + \max_{0 \leq i < n} \left| \frac{a_i}{a_n} \right|.$$

To avoid confusion with the notation $\mathbb{R}$ for the real numbers, we denote this bound by $B(P)$. Therefore, all real roots of P lie in the interval:

$$[-B(P), B(P)].$$

Cauchy-type root bounds are discussed in [1, Chapter 10.1, Notation 10.1 and Lemma 10.2]. We also keep the web reference originally used in the project for general background on root bounds [3]. This interval is used as the initial search domain in the implemented algorithms.

2.6 Sturm–Habicht Sequences

The Sturm–Habicht sequence is a generalization of the classical Sturm sequence based on subresultant theory. It is constructed from subresultant polynomials together with suitable sign corrections [2].

A key fact for this project is that the Sturm–Habicht sequence provides the same root-counting information as the Sturm sequence. In the paper of Gonzalez, Lombardi, Recio, and Roy, this correspondence is formalized in [2, Theorem 4], which shows that the sign variations of the Sturm–Habicht coefficients recover the same counting quantity as the Sturm sequence.

This result is the theoretical justification for the second algorithm implemented in this report.

3 Algorithms

3.1 Notation for Algorithmic Construction

For a polynomial $P \in \mathbb{Q}[x]$, we use the following notation:

- $\text{lcoef}(P)$: the leading coefficient of P ;
- $\text{rem}(A, B)$: the remainder of the Euclidean division of A by B ;
- $\text{Sres}_j(\cdot)$: the j -th subresultant polynomial;
- $\delta_k = (-1)^{k(k+1)/2}$.

3.2 Construction of the Sturm Sequence

Algorithm 1 Construction of the Sturm Sequence

```

1: Input:  $P \in \mathbb{Q}[x]$  with  $\deg(P) = p$ 
2: Output:  $(S_0, \dots, S_t)$ 
3:  $S_0 \leftarrow P, S_1 \leftarrow P', i \leftarrow 1$ 
4: while  $i < p$  do
5:    $S_{i+1} \leftarrow -\text{rem}(S_{i-1}, S_i)$ 
6:   if  $\deg(S_{i+1}) = 0$  then
7:     break
8:    $i \leftarrow i + 1$ 
9: return  $(S_0, \dots, S_{i+1})$ 

```

3.3 Construction of the Sturm–Habicht Sequence

Algorithm 2 Construction of the Sturm–Habicht Sequence

```

1: Input:  $P \in \mathbb{Q}[x] \setminus \{0\}$  with  $\deg(P) = p$ 
2: Output:  $(H_0, \dots, H_p)$ 
3:  $L \leftarrow \text{lcoef}(P), D \leftarrow P'$ 
4: if  $L = 1$  then
5:    $R \leftarrow \text{rem}(D, P), r \leftarrow \deg(R)$ 
6:    $H_p \leftarrow P, H_{p-1} \leftarrow R$ 
7:   if  $R \neq 0$  then
8:      $H_r \leftarrow \text{lcoef}(R)^{p-r+1} R$ 
9:     for  $j \leftarrow r + 1$  to  $p - 2$  do
10:       $H_j \leftarrow 0$ 
11:     for  $j \leftarrow 0$  to  $r - 1$  do
12:       $H_j \leftarrow \delta_{p-j-1} \text{Sres}_j(P, p, R, r)$ 
13:   else
14:     for  $j \leftarrow 0$  to  $p - 2$  do
15:       $H_j \leftarrow 0$ 
16:   else
17:      $H_p \leftarrow LP, H_{p-1} \leftarrow LD$ 
18:     for  $j \leftarrow 0$  to  $p - 2$  do
19:       $H_j \leftarrow \delta_{p-j-1} \text{Sres}_j(P, p, D, p - 1) / L$ 
20: return  $(H_0, \dots, H_p)$ 

```

3.4 Unified Root Isolation Procedure

Both methods use the same recursive subdivision scheme. The only difference is the sequence used to compute sign variations at interval endpoints: the naive

method uses the Sturm sequence, while the Sturm–Habicht method uses the Sturm–Habicht sequence.

Algorithm 3 Root Isolation by Sign Variations

```

1: Input:  $P \in \mathbb{Q}[x]$  and an associated sequence  $\mathcal{S} = (S_0, \dots, S_t)$ 
2: Output:  $\mathcal{I}$  (isolating intervals)
3: Compute  $B(P)$  and construct  $\mathcal{S}$ 
4:  $\mathcal{I} \leftarrow \emptyset$ , stack  $\leftarrow \{[-B(P), B(P)]\}$ 
5: while stack not empty do
6:   extract  $[a, b]$ 
7:    $k \leftarrow V_{\mathcal{S}}(a) - V_{\mathcal{S}}(b)$ 
8:   if  $k = 0$  then
9:     discard  $[a, b]$ 
10:  else if  $k = 1$  then
11:     $\mathcal{I} \leftarrow \mathcal{I} \cup \{[a, b]\}$ 
12:  else
13:    subdivide  $[a, b]$  and push the subintervals onto the stack
14: return  $\mathcal{I}$ 

```

Instantiation. Choosing $\mathcal{S}$ as the classical Sturm sequence gives the naive Sturm method; choosing $\mathcal{S}$ as the Sturm–Habicht sequence gives the Sturm–Habicht method.

Justification. If $\mathcal{S}$ is the Sturm sequence, correctness follows from Sturm’s theorem, namely [1, Theorem 2.50], which states that the difference of sign variations at the endpoints equals the number of real roots in the interval. If $\mathcal{S}$ is the Sturm–Habicht sequence, correctness follows from [2, Theorem 4], which shows that the Sturm–Habicht sequence provides the same root-counting information. Hence the same decision rule applies in both cases. Since the number of real roots is finite and the intervals shrink during subdivision, the procedure terminates and isolates all real roots.

3.5 Comparison of the Two Approaches

Both methods use the same isolation strategy; they differ only in the construction of the sequence used for sign-variation computations.

The Sturm sequence is based on Euclidean remainders and is simpler to implement. The Sturm–Habicht sequence relies on subresultants and determinant computations, making it more involved but better suited to exact arithmetic. In practice, it is expected to perform better in difficult cases, for example when roots are very close.

3.6 Choice of the Subdivision Parameter

In the recursive isolation procedure, an interval containing several roots is subdivided into `nbI` smaller subintervals. Although any fixed subdivision rule ensures correctness, the value of `nbI` affects performance: a small value increases the recursion depth, while a large value increases the number of endpoint evaluations.

We therefore performed preliminary tests with `nbI` ranging from 2 to 10 on several random instances using the Sturm–Habicht method. In our experiments,

$$\text{nbI} = 3$$

gave the best overall compromise. For this reason, all experiments reported below were carried out with `nbI=3`.

<code>nbI</code>	(40, [−50, 50])	(40, [−50, 50])	(40, [−100, 100])	(30, [−100, 100])
2	98.177246	89.687576	132.237703	23.679644
3	57.989370	50.613176	73.630933	14.243418
4	121.293385	88.026356	133.968599	24.658779
5	79.633296	57.497845	84.971422	15.555658
6	146.454914	103.250890	154.838307	27.758197
7	98.457046	69.710708	99.421169	18.130980
8	164.484076	118.062334	175.542970	33.040201
9	109.823818	77.170983	109.985968	21.256938
10	187.272005	139.524913	201.870030	40.214251

Table 1: Preliminary timing results for different values of the subdivision parameter `nbI` with the Sturm–Habicht method.

4 Implementation Details

The source code of the project is available at: <https://github.com/ZHJ0022/sorbonne.info.m1s2.pcca2>

The project is implemented in C and relies on exact rational arithmetic. In order to avoid numerical errors during polynomial manipulations, coefficients are represented with the GMP type `mpq_t`. For some operations involving polynomial evaluation and determinants, the implementation also uses the FLINT library, in particular the types `fmpq_poly_t` and `fmpq_mat_t`.

4.1 General Data Structures

The main data structure is the `Polynomial` structure defined in `include/pcca2.h`. A polynomial is represented by:

- an array `coeffs` of rational coefficients, where `coeffs[i]` stores the coefficient of x^i ;

- an integer `degree`;
- an optional array `roots`, used in the test phase to store known roots for verification.

4.2 Memory Management and Auxiliary Functions

The file `src/util.c` contains the auxiliary routines used throughout the project. The function `create_polynomial` allocates a polynomial of a given degree and initializes all coefficients to zero. The function `free_polynomial` releases both the coefficients and the optional stored roots. A deep copy operation is provided by `copy_polynomial`.

The same file also includes `generate_random_polynomial`, which constructs a polynomial from randomly generated rational roots in a prescribed interval. This is useful for testing the correctness of the root isolation algorithms, since the true roots are known in advance. Finally, the function `afficher_bound` prints the isolated intervals returned by the algorithms.

4.3 Basic Polynomial Operations

The basic algebraic operations are implemented in `src/poly.c`. The derivative is computed by `poly_derivative`. Polynomial multiplication is implemented by the naive `poly_mul_naive`. The Euclidean remainder is computed by `poly_remainder`, which is one of the key ingredients of the naive Sturm sequence.

Several utility operations are also implemented:

- `poly_trim_degree`, which removes trailing zero coefficients and updates the degree;
- `poly_negative`, which multiplies a polynomial by -1 ;
- `poly_get_lc`, which extracts the leading coefficient;
- `poly_is_zero` and `poly_is_monic`, which test structural properties of a polynomial;
- `poly_scalar_mul` and `poly_scalar_div`, which scale all coefficients by a rational factor.

Polynomial evaluation is implemented in two ways. The function `poly_calculate_sign` uses Horner's method directly on GMP rationals. In addition, the pair `polynomialConv` and `poly_sign_flint` converts the polynomial to FLINT format and evaluates it using FLINT routines. In the recursive isolation step, the implementation uses the FLINT-based evaluation in order to compute sign variations efficiently.

4.4 Root Bounds and Sign Variations

The file `src/calculate.c` gathers the elementary numerical routines needed by both isolation methods.

The function `cauchy_bound` computes the classical Cauchy bound

$$R = 1 + \max_{0 \leq i < n} \left| \frac{a_i}{a_n} \right|,$$

which guarantees that all real roots lie in the interval $[-R, R]$. This interval is used as the starting point for the recursive subdivision procedure.

The function `nb_sign_change` computes the number of sign changes in a sequence while ignoring zeros, which is exactly the quantity needed in Sturm's theorem. The file also contains `verify_interval`, a testing function that checks whether every known root of a generated polynomial belongs to one of the isolated intervals. Finally, the helper `delta_sign` computes the factor $(-1)^{k(k+1)/2}$ used in the construction of the Sturm–Habicht sequence.

4.5 Naive Sturm Sequence Implementation

The naive implementation of Sturm's method is contained in `src/sturmNaif.c`. The function `sturm_naif` first constructs the Sturm sequence:

$$S_0 = P, \quad S_1 = P', \quad S_{i+1} = -\text{rem}(S_{i-1}, S_i).$$

The construction stops when a constant polynomial is reached.

Once the sequence has been built, the algorithm computes a global search interval using the Cauchy bound and calls the recursive function `bound_recu`. This function evaluates the whole Sturm sequence at the endpoints of a subinterval, computes the number of sign variations at both ends, and deduces the number of roots in the interval from their difference.

The recursive strategy is the following:

- if an interval contains no root, it is discarded;
- if it contains exactly one root, it is returned as an isolating interval;
- otherwise, the interval is subdivided into smaller subintervals and the process is applied recursively.

The output format is an array of rational values: the first entry stores the number of isolated real roots, and the remaining entries store the interval endpoints.

4.6 Subresultants and Sylvester Matrices

The file `src/subresultant.c` implements the subresultant machinery needed for the Sturm–Habicht approach. First, the code allocates rational matrices with

`alloc_matrix`. The function `create_sylv_j_matrix` constructs the truncated Sylvester matrix associated with two polynomials P and S and an index j .

The determinant computations are performed by FLINT. Since the rest of the project uses GMP rationals, the code provides two conversion functions, `mpq_to_fmpq` and `fmpq_to_mpq`, to move between the two libraries. The function `sylv_j_det` extracts the appropriate square minor and computes its determinant. From these determinants, the function `subresultant_j` reconstructs the j -th subresultant polynomial.

This part of the project is more algebraically involved than the naive Sturm implementation, since it explicitly manipulates Sylvester matrices and determinant-based constructions.

4.7 Sturm–Habicht Sequence Implementation

The file `src/sturmHabicht.c` implements the second root isolation method. The function `create_sturm_habicht` constructs the full Sturm–Habicht sequence of a polynomial. The implementation distinguishes two main cases:

- the case where the polynomial is monic, where the sequence starts from P and the remainder of the division of P' by P ;
- the case where the leading coefficient is not equal to 1, where additional scaling by the leading coefficient is required.

The sequence is completed using subresultants together with the sign factor $(-1)^{k(k+1)/2}$. Once the sequence has been constructed, the function `sturm_habicht` applies the same recursive isolation routine `bound_recu` as in the naive Sturm method.

5 Experimental Protocol and Validation

5.1 Test Instances

In order to validate the implementation, we consider polynomials generated by the function `generate_random_polynomial`. This function constructs a polynomial from randomly generated rational roots within a prescribed interval.

This approach provides a convenient testing framework, since the exact real roots of the polynomial are known in advance. As a result, it becomes possible to directly compare the computed isolating intervals with the true roots.

Each test instance is processed independently by both the naive Sturm method or the Sturm–Habicht method.

5.2 Validation of Isolating Intervals

The correctness of the computed isolating intervals is verified using the function `verify_interval`.

Given a polynomial with known roots, and a list of intervals returned by the isolation algorithm, the validation proceeds as follows:

- for each known root, check whether it belongs to at least one of the computed intervals;
- ensure that each interval contains a root;
- verify that the intervals are disjoint.

This verification step guarantees that the output satisfies the definition of real root isolation: every real root is contained in an interval, and each interval isolates exactly one root.

5.3 Evaluation Criteria

The implemented methods are evaluated according to two main criteria.

Correctness. The primary objective is to ensure that both algorithms correctly isolate all real roots. This is assessed through the validation procedure described above, by comparing the computed intervals with the known roots of the test polynomials.

Execution Time. The computational efficiency of the algorithms is also considered. We compare the relative execution time of the naive Sturm method and the Sturm–Habicht method on the same test instances. This comparison provides insight into the practical cost of the more advanced algebraic constructions used in the Sturm–Habicht approach.

6 Tests and Results

6.1 Structured Polynomials

As a first family of experiments, we considered the structured polynomials

$$P_n(x) = \prod_{k=1}^n (x - k),$$

whose real roots are the integers $1, \dots, n$. These tests were implemented in `test1.c`. For each value of n , we measured:

- the time needed to construct the Sturm sequence;
- the time needed to compute the isolating intervals with the naive Sturm method;
- the time needed to construct the Sturm–Habicht sequence;

- the time needed to compute the isolating intervals with the Sturm–Habicht method.

All tested instances were correctly solved by both methods.

Polynomial	Naive Sturm: sequence (s)	Naive Sturm: intervals (s)	Sturm–Habicht: sequence (s)	Sturm–Habicht: intervals (s)
$(x - 1) \cdots (x - 5)$	0.000044	0.000638	0.000551	0.000146
$(x - 1) \cdots (x - 10)$	0.000049	0.001940	0.000393	0.001599
$(x - 1) \cdots (x - 15)$	0.000068	0.009779	0.004573	0.009929
$(x - 1) \cdots (x - 20)$	0.000120	0.037683	0.036871	0.036150
$(x - 1) \cdots (x - 25)$	0.000174	0.102967	0.182652	0.102565
$(x - 1) \cdots (x - 30)$	0.000311	0.273682	0.588022	0.273682

Table 2: Execution times for structured polynomials $P_n(x) = \prod_{k=1}^n (x - k)$.

Table 2 shows that both methods correctly isolate the real roots on all tested examples. The naive Sturm method is systematically faster for the construction of the sequence, while the interval computation times are very close for the two methods.

This behavior is consistent with the structure of the algorithms. The naive Sturm sequence is obtained by repeated Euclidean remainders, whereas the Sturm–Habicht sequence requires subresultant computations, which are algebraically more expensive. In our implementation, both methods then use the same recursive subdivision procedure, which helps explain why the interval computation times are of the same order on these examples.

For this family of examples, the roots are well separated and regularly distributed. As a consequence, the isolation step is not especially difficult, and the extra cost of constructing the Sturm–Habicht sequence is not compensated by a significant gain during subdivision. In this regime, the naive Sturm method is therefore more efficient overall.

6.2 Random Polynomials

In order to evaluate the two methods on less structured inputs, we generated random polynomials of degree 40 whose roots lie in the interval $[-100, 100]$. These experiments were implemented in `test3.c`. For each random instance, we measured:

- the time needed to generate the polynomial;
- the time needed to construct the Sturm sequence;
- the time needed to compute the isolating intervals with the naive Sturm method;
- the number of evaluations performed by the naive Sturm method;

- the time needed to construct the Sturm–Habicht sequence;
- the time needed to compute the isolating intervals with the Sturm–Habicht method;
- the number of evaluations performed by the Sturm–Habicht method.

We repeated the experiment on five independently generated random instances, using the same subdivision parameter `nbI` = 3. In all cases, both methods returned correct isolating intervals.

Test	Poly. gen. time (s)	Naive Sturm: sequence (s)	Naive Sturm: intervals (s)	Naive eval. count	Sturm–Habicht: sequence (s)	Sturm–Habicht: intervals (s)	Habicht eval. count
Random #1	0.000339	1.388559	74.605602	37966	7.812085	3.223181	37966
Random #2	0.000342	1.210464	79.087875	42640	7.737090	3.613999	42640
Random #3	0.000326	0.999167	60.887627	39934	7.415293	3.050134	39934
Random #4	0.000409	1.257352	79.696760	41656	7.887526	3.764294	41656
Random #5	0.000406	1.608159	96.909536	42394	9.619900	4.621253	42394

Table 3: Execution times and evaluation counts for five random polynomials of degree 40 with roots in $[-100, 100]$.

The results in Table 3 show a clear contrast between the two phases of the computation. As already observed on structured examples, the naive Sturm method is faster for sequence construction. However, this advantage is outweighed by the much faster interval computation obtained with Sturm–Habicht.

For all five random instances, the interval computation takes about 3 to 5 seconds with Sturm–Habicht, whereas the corresponding time for the naive Sturm method ranges from about 61 to 97 seconds. Thus, even though the Sturm–Habicht sequence is more expensive to construct, the total running time is significantly smaller for the Sturm–Habicht method.

The evaluation counts help to clarify this difference. For each random instance, both methods perform exactly the same number of evaluations, since they use the same recursive subdivision procedure in our implementation. Therefore, the difference in interval time mainly reflects the cost of evaluating the corresponding sequence at the interval endpoints, rather than a difference in the number of explored intervals.

This behavior is consistent with the theoretical discussion: the Sturm–Habicht sequence is more costly to construct because it is based on subresultants rather than direct Euclidean remainders, but this higher construction cost can be compensated by a much faster evaluation phase during recursive root isolation.

To further illustrate this phenomenon, we also tested a larger random instance in `test4.c`, corresponding to a polynomial of degree 80 with roots in $[-1000, 1000]$. In this case, the naive Sturm method did not finish within one hour and was stopped manually, while the Sturm–Habicht method completed the computation in 1901.705380 seconds (approximately 31.7 minutes). This large-scale example confirms that, for harder random instances, the practical advantage of the Sturm–Habicht approach becomes even more significant.

Polynomial	Naive Sturm (total time)	Sturm–Habicht (total time)
Random degree 80, roots in $[-1000, 1000]$	> 1 hour	1901.705380 s

Table 4: Large random instance illustrating the scalability gap between the two methods.

This additional experiment is not meant to provide a statistical average, but rather to highlight the asymptotic tendency already visible in Table 3: for sufficiently large random inputs, the naive Sturm method becomes impractical, while the Sturm–Habicht method remains feasible.

6.3 Close-Root Polynomials

In order to complement the structured and random test families, we also considered polynomials whose real roots are deliberately chosen to be very close to each other. The purpose of this experiment is to construct instances that are more challenging for root isolation, since very small separations between consecutive roots may force the subdivision process to examine narrower intervals before each root is isolated.

More precisely, for fixed integers $d \geq 1$ and $k \geq 1$, we first considered the regular family

$$P_{d,k}(x) = \prod_{i=0}^{d-1} (2^k x - i),$$

whose roots are

$$\left\{ \frac{i}{2^k} \mid 0 \leq i \leq d-1 \right\}.$$

The distance between two consecutive roots is exactly 2^{-k} , so increasing k produces increasingly clustered roots. However, this deterministic family is also very regular. In preliminary tests, both methods remained relatively fast, and the naive Sturm method could even be more advantageous overall because the gain during isolation was not large enough to compensate for the construction cost of the Sturm–Habicht sequence.

For this reason, we modified the experiment to obtain less regular close-root instances. Instead of taking exactly the roots $i/2^k$, we generated one random root inside each small interval

$$\left(\frac{i}{2^k}, \frac{i+1}{2^k} \right), \quad 0 \leq i \leq d-1.$$

Thus, the tested polynomials have the form

$$\tilde{P}_{d,k}(x) = \prod_{i=0}^{d-1} (x - \alpha_i),$$

where

$$\alpha_i \in \left(\frac{i}{2^k}, \frac{i+1}{2^k} \right).$$

In the implementation, these random roots were generated using floating-point values and then converted to rational coefficients. Consequently, the roots remain close to the grid points $i/2^k$, but the resulting polynomial is no longer as regular as the deterministic product.

In the experiments below, we fixed $d = 40$, used the subdivision parameter `nbI` = 3, and tested the values $k = 16, 24, 32$. All instances were solved correctly by both methods.

Polynomial	Naive Sturm: sequence (s)	Naive Sturm: intervals (s)	Sturm–Habicht: sequence (s)	Sturm–Habicht: intervals (s)	Evaluation count
$k = 16$	15.527218	253.239450	182.089287	3.384850	10168
$k = 24$	17.900251	310.183530	252.472881	4.039648	11152
$k = 32$	18.165320	339.249699	285.508664	5.912320	12382

Table 5: Execution times for randomized close-root polynomials of degree 40, with one root in each interval $(i/2^k, (i+1)/2^k)$.

The results in Table 5 show that the trade-off observed on random polynomials becomes even more pronounced. The naive Sturm method is still much faster for sequence construction: its construction time stays between about 15 and 18 seconds, whereas the Sturm–Habicht construction time increases from about 182 seconds for $k = 16$ to more than 285 seconds for $k = 32$.

However, the interval-computation phase shows the opposite behaviour. The naive Sturm method requires several hundreds of seconds to compute the isolating intervals, from about 253 seconds to about 339 seconds. By contrast, the Sturm–Habicht method only needs a few seconds during the same phase.

As in the previous subsection, both methods use the same recursive subdivision procedure in our implementation. Since the evaluation counts are identical in each test, the large difference in interval time mainly reflects the cost of evaluating the corresponding sequence at the interval endpoints. In these randomized close-root instances, this effect becomes especially pronounced.

This difference is large enough to compensate for the higher construction cost of the Sturm–Habicht sequence. For example, for $k = 32$, the total time is approximately

$$18.165320 + 339.249699 = 357.415019 \text{ seconds}$$

for the naive Sturm method, whereas it is approximately

$$285.508664 + 5.912320 = 291.420984 \text{ seconds}$$

for the Sturm–Habicht method. Thus, although Sturm–Habicht is more expensive to construct, it is faster overall on these randomized close-root instances.

Conclusion. These experiments show that close-root instances must be interpreted with care. The fully regular family

$$\prod_{i=0}^{d-1} (2^k x - i)$$

does not necessarily reveal the practical difficulty of the isolation phase, because its special algebraic structure may make the computation comparatively easy. By contrast, when one random root is chosen in each small interval

$$\left(\frac{i}{2^k}, \frac{i+1}{2^k} \right),$$

the roots remain very close to each other, but the polynomial loses this regular structure.

In this randomized close-root setting, the naive Sturm method remains much cheaper for sequence construction, but its interval computation becomes very expensive. The Sturm–Habicht method has the opposite profile: its construction phase is costly, but its evaluation during recursive subdivision is much faster. Since the gain in the interval phase is larger than the loss in the construction phase, the Sturm–Habicht method is more efficient overall for the close-root polynomials considered here.

References

- [1] Saugata Basu, Richard Pollack, and Marie-Françoise Roy. *Algorithms in Real Algebraic Geometry*. 2nd ed. Vol. 10. Algorithms and Computation in Mathematics. Springer, 2006.
- [2] Laureano Gonzalez et al. “Sturm-Habicht Sequence”. In: *Proceedings of the ACM-SIGSAM 1989 International Symposium on Symbolic and Algebraic Computation (ISSAC ’89)*. ACM, 1989, pp. 136–145.
- [3] Wikipedia contributors. *Geometrical properties of polynomial roots*. https://en.wikipedia.org/wiki/Geometrical_properties_of_polynomial_roots. Original web reference used in the project. n.d.